



École Marocaine des Sciences de l'Ingénieur

Filière : Ingénierie Informatique et Réseaux

Option : Intelligence Artificielle et Data Science

4^e année Génie Informatique

Module : Deep Learning

Rapport de Projet de Fin de Module

MusicAI

**Modèles de Deep Learning pour données tabulaires,
images et séquences musicales**

Spotify Tracks, GTZAN Mel-spectrogrammes et MAESTRO MIDI

Réalisé par : Taha ZAAMI

Encadré par : Pr. BATAL Mossab

Année Universitaire 2025–2026

Résumé Exécutif

Ce rapport présente une étude comparative de trois familles fondamentales du Deep Learning appliquées au domaine musical : le perceptron multicouche (MLP), les réseaux de neurones convolutifs (CNN) et les modèles séquentiels RNN/LSTM/GRU avec une extension Seq2Seq. L'objectif est de montrer que le choix d'une architecture ne dépend pas uniquement de la performance recherchée, mais surtout de la structure des données étudiées.

La partie MLP exploite le dataset Spotify Tracks sous forme tabulaire afin de classifier des genres musicaux à partir de caractéristiques audio numériques. La partie CNN transforme les fichiers audio du dataset GTZAN en Mel-spectrogrammes, assimilés à des images, pour étudier l'intérêt des convolutions. La partie séquentielle utilise le dataset MAESTRO MIDI afin de prédire et générer des suites de notes musicales.

La démarche adoptée suit une logique théorique et expérimentale. Chaque chapitre présente d'abord les fondements du modèle, puis le cadre expérimental, les résultats obtenus avec PyTorch, leur interprétation critique et une synthèse. Les expériences montrent que le MLP est pertinent pour des vecteurs numériques bien préparés, que le CNN exploite mieux la structure spatiale des spectrogrammes, et que les modèles récurrents permettent de représenter la dépendance temporelle dans des séquences MIDI.

Table des matières

Résumé Exécutif	1
Introduction Générale	6
I Le Perceptron Multicouche (MLP)	7
I.1 Fondements Théoriques et Mathématiques	7
I.1.1 Architecture et propagation avant	7
I.1.2 Fonction de perte et optimisation	7
I.1.3 Rétropropagation et paramètres PyTorch	7
I.1.4 Stratégies d'initialisation	7
I.2 Cadre Expérimental	8
I.3 Étude des Stratégies d'Initialisation	8
I.4 Comparaison : nn.Sequential vs Classe Personnalisée	9
I.5 Évaluation Finale sur le Test Set	9
I.6 Synthèse du Chapitre I	10
II Réseaux de Neurones Convolutifs (CNN)	11
II.1 Fondements Théoriques et Mathématiques	11
II.1.1 Limites du MLP sur les images	11
II.1.2 Corrélation croisée 2D	11
II.1.3 Formule dimensionnelle	11
II.1.4 Pooling et convolution 1×1	11
II.2 Cadre Expérimental	11
II.3 Étude des Variantes Architecturales	12
II.4 Visualisation des Feature Maps	13
II.5 Évaluation Finale	14
II.6 Synthèse du Chapitre II	15
III Réseaux Récurrents, LSTM, GRU et Seq2Seq	16
III.1 Fondements Théoriques et Mathématiques	16
III.1.1 Limites des architectures statiques face aux séquences	16
III.1.2 RNN simple	16

III.1.3 BPTT et gradient clipping	16
III.1.4 LSTM et GRU	16
III.1.5 Architecture Seq2Seq	16
III.2 Cadre Expérimental	17
III.3 Comparaison des Cellules Récurrentes	17
III.4 Génération Musicale Simple	18
III.5 Mini Système Seq2Seq	18
III.6 Synthèse du Chapitre III	19
IV Comparaison Croisée et Conclusion Générale	20
IV.1 Tableau Comparatif Global des Trois Architectures	20
IV.2 Le Biais Inductif : Clé de la Réussite en Deep Learning	20
IV.3 Perspectives d'Amélioration	20
IV.4 Conclusion Générale	21
Références Bibliographiques	22

Table des figures

1	Comparaison des accuracies de validation des modèles MLP	9
2	Courbes d'entraînement du meilleur MLP personnalisé avec initialisation Xavier	9
3	Matrice de confusion du meilleur MLP sur le test set Spotify	10
1	Exemples de Mel-spectrogrammes générés à partir du dataset GTZAN	12
2	Comparaison des accuracies de validation des modèles CNN	13
3	Visualisation de cartes de caractéristiques apprises par la première convolution du CNN	14
4	Matrice de confusion du meilleur modèle CNN sur GTZAN	14
1	Comparaison de la perplexité de validation des modèles séquentiels	17
2	Courbes d'entraînement du modèle LSTM	18
3	Courbes d'entraînement du modèle Seq2Seq	19

Liste des tableaux

1	Paramètres expérimentaux du Chapitre I – MLP / Spotify Tracks	8
2	Résultats comparatifs des expériences MLP	8
1	Paramètres expérimentaux du Chapitre II – CNN / GTZAN	12
2	Résultats comparatifs des modèles image sur le set de validation	12
1	Paramètres expérimentaux du Chapitre III – RNN/LSTM/GRU/Seq2Seq	17
2	Comparaison RNN, LSTM et GRU sur la prédiction de notes	17
1	Comparaison globale des architectures étudiées dans MusicAI	20

Introduction Générale

L'apprentissage profond ne repose pas sur un modèle unique capable de traiter efficacement tous les types de données. Au contraire, il s'appuie sur des architectures spécialisées qui exploitent des hypothèses différentes sur la structure des observations. Ces hypothèses sont appelées biais inductifs. Un MLP traite une observation comme un vecteur de caractéristiques, un CNN suppose une organisation spatiale locale, tandis qu'un RNN exploite l'ordre temporel d'une séquence.

Dans ce projet de fin de module, le fil conducteur choisi est la musique. Cette thématique permet de manipuler trois types de données complémentaires : des données tabulaires Spotify, des spectrogrammes Mel construits à partir de fichiers audio GTZAN, et des séquences de notes extraites de fichiers MIDI MAESTRO. Le projet permet ainsi de relier les notions théoriques du module de Deep Learning à une implémentation pratique sous PyTorch dans Google Colab.

La problématique générale peut être formulée comme suit : comment adapter le choix d'une architecture de Deep Learning à la nature des données étudiées, et comment interpréter expérimentalement les performances obtenues ? Pour répondre à cette question, le rapport est structuré en quatre chapitres. Le premier chapitre traite le MLP sur données tabulaires. Le deuxième chapitre étudie les CNN pour les spectrogrammes. Le troisième chapitre analyse les modèles RNN, LSTM, GRU et Seq2Seq pour les séquences MIDI. Le quatrième chapitre propose une comparaison croisée et une conclusion générale.

Chapitre I

Le Perceptron Multicouche (MLP)

I.1 Fondements Théoriques et Mathématiques

I.1.1 Architecture et propagation avant

Un perceptron multicouche est un réseau feed-forward composé d'une couche d'entrée, d'une ou plusieurs couches cachées et d'une couche de sortie. Pour une couche l , la propagation avant s'écrit :

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \sigma(z^{(l)}).$$

Dans ce projet, les couches cachées utilisent principalement l'activation ReLU :

$$\text{ReLU}(z) = \max(0, z).$$

Le MLP ne possède aucun biais spatial ou temporel explicite : il apprend uniquement à partir des variables tabulaires fournies.

I.1.2 Fonction de perte et optimisation

Pour une classification multiclasse, la fonction de perte utilisée est la Cross-Entropy Loss :

$$\mathcal{L}(y, \hat{y}) = - \sum_{c=1}^C y_c \log(\hat{y}_c).$$

L'optimisation est réalisée avec Adam, qui combine une moyenne mobile du gradient et une moyenne mobile du carré du gradient afin d'adapter le taux d'apprentissage à chaque paramètre.

I.1.3 Rétropropagation et paramètres PyTorch

Dans PyTorch, un modèle est défini comme une classe héritant de `nn.Module` ou comme une séquence de couches avec `nn.Sequential`. Les poids et biais sont enregistrés comme paramètres apprenables. Après un passage avant, l'appel à `loss.backward()` calcule les gradients, puis l'optimiseur met à jour les paramètres. La sauvegarde du `state_dict` permet de recharger uniquement les poids du meilleur modèle.

I.1.4 Stratégies d'initialisation

Trois stratégies sont testées : l'initialisation gaussienne, l'initialisation constante et l'initialisation Xavier. L'initialisation constante est utile pédagogiquement, mais elle provoque un problème de symétrie entre neurones. Xavier est généralement plus stable car elle adapte la variance au nombre d'entrées et de sorties.

I.2 Cadre Expérimental

Le dataset utilisé est Spotify Tracks Dataset. Chaque observation représente une musique décrite par des caractéristiques numériques telles que `danceability`, `energy`, `valence`, `tempo`, `loudness` ou `acousticness`. La colonne cible retenue est `track_genre`. Comme le dataset contient un grand nombre de genres, seuls les dix genres les plus fréquents sont conservés.

Table 1 – Paramètres expérimentaux du Chapitre I – MLP / Spotify Tracks

Paramètre	Valeur / Description
Dataset	Spotify Tracks Dataset
Tâche	Classification de genres musicaux
Échantillons	10 000 morceaux après conservation des 10 genres les plus fréquents
Variables d'entrée	11 caractéristiques audio normalisées
Classes	acoustic, afrobeat, alt-rock, alternative, ambient, anime, black-metal, bluegrass, blues, brazil
Split	70% apprentissage, 15% validation, 15% test
Modèles	MLP avec <code>nn.Sequential</code> et MLP personnalisé <code>nn.Module</code>
Initialisations	Gaussienne, constante et Xavier
Métriques	Accuracy, précision macro, rappel macro, F1-score macro, matrice de confusion

I.3 Étude des Stratégies d'Initialisation

Table 2 – Résultats comparatifs des expériences MLP

Expérience	Val. loss	Val. accuracy	Epochs
MLPSequential_gaussian	1.0264	64.13%	20
MLPSequential_constant	1.0557	62.53%	20
MLPSequential_xavier	1.0137	63.67%	20
MLPCustom_gaussian	1.0451	63.80%	20
MLPCustom_constant	1.0642	63.07%	20
MLPCustom_xavier	1.0109	64.73%	20

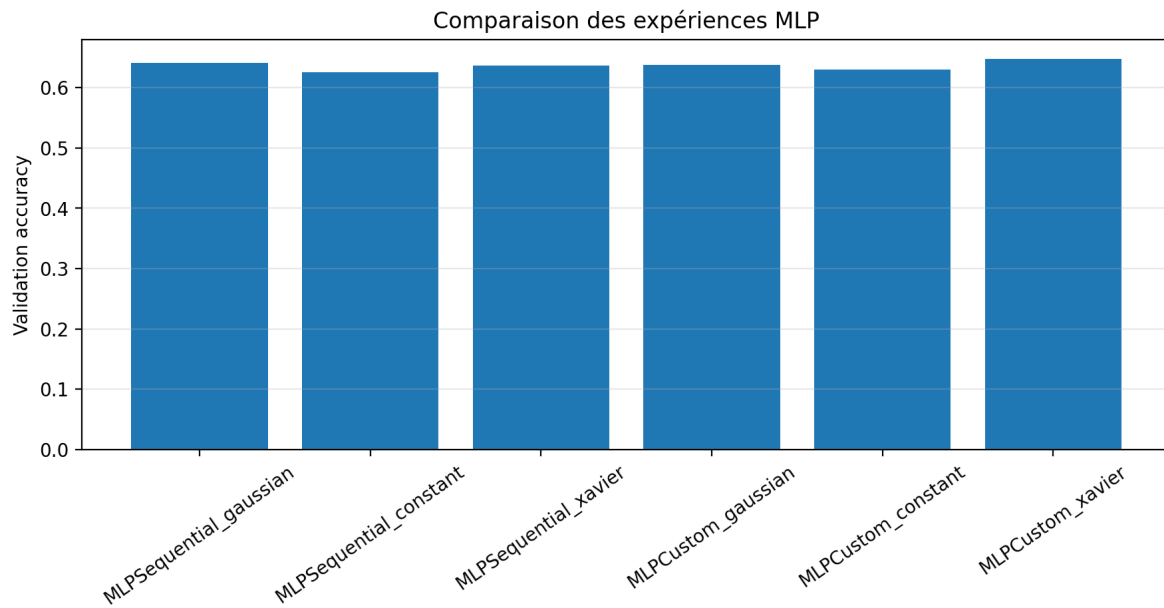


Figure 1 – Comparaison des accuracies de validation des modèles MLP

I.4 Comparaison : nn.Sequential vs Classe Personnalisée

Le modèle `nn.Sequential` permet de construire rapidement une chaîne linéaire de couches. La classe personnalisée héritant de `nn.Module` offre davantage de flexibilité pour définir le passage avant, ajouter des branchements ou inspecter plus clairement les paramètres.

Le meilleur résultat est obtenu par le modèle `MLPCustom` avec initialisation Xavier. Cela montre que la classe personnalisée n'est pas seulement utile pour le code, mais aussi pour structurer plus proprement l'expérimentation et l'inspection des paramètres.



Figure 2 – Courbes d'entraînement du meilleur MLP personnalisé avec initialisation Xavier

I.5 Évaluation Finale sur le Test Set

Le meilleur modèle rechargé par `state_dict` obtient une loss de test de 1.0155, une accuracy de 61.60%, une précision macro de 61.60%, un rappel macro de 61.60% et un F1-score macro de 60.80%.

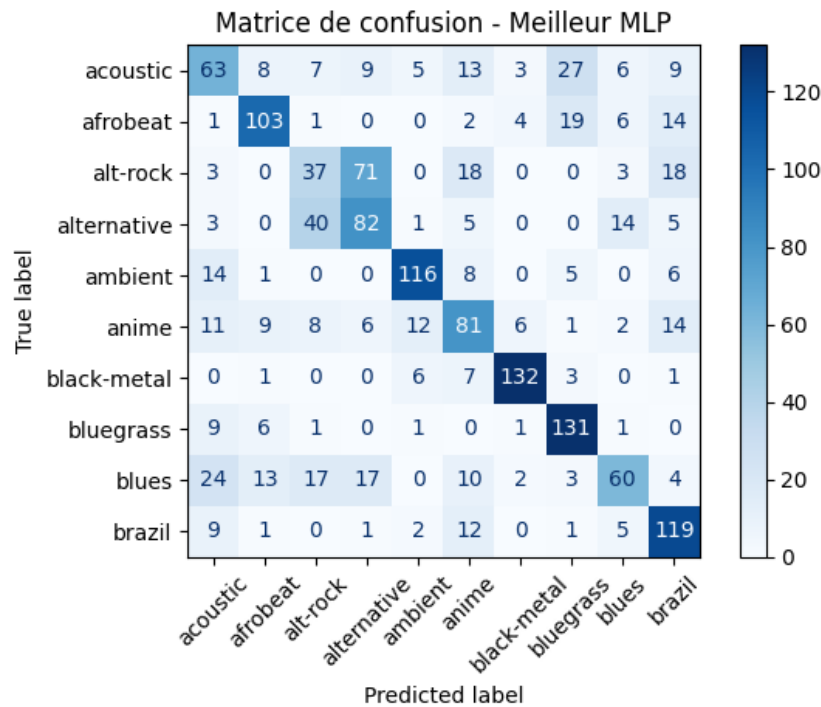


Figure 3 – Matrice de confusion du meilleur MLP sur le test set Spotify

Analyse et interprétation

Le MLP constitue une solution pertinente pour des données tabulaires, mais ses performances dépendent fortement de la qualité des variables fournies. Dans cette expérience, certaines classes comme *black-metal*, *ambient* ou *bluegrass* sont mieux distinguées, tandis que des genres proches comme *alt-rock*, *alternative* ou *blues* produisent davantage de confusions. L'initialisation Xavier donne le meilleur résultat, ce qui confirme son intérêt pour stabiliser la propagation du signal.

I.6 Synthèse du Chapitre I

- Le MLP est adapté aux données tabulaires préparées sous forme de vecteurs.
- `nn.Sequential` est simple et efficace pour un pipeline linéaire.
- `nn.Module` devient indispensable pour les architectures plus flexibles.
- L'initialisation et la normalisation influencent fortement la stabilité de l'apprentissage.
- Les limites observées proviennent surtout de la proximité statistique entre certains genres musicaux.

Chapitre II

Réseaux de Neurones Convolutifs (CNN)

II.1 Fondements Théoriques et Mathématiques

II.1.1 Limites du MLP sur les images

Un MLP appliqué à une image doit d'abord aplatir celle-ci en un vecteur. Cette opération détruit la topologie spatiale : deux pixels voisins dans l'image peuvent devenir simplement deux valeurs indépendantes dans le vecteur. Le MLP doit donc apprendre les relations spatiales sans biais inductif adapté.

II.1.2 Corrélation croisée 2D

Un CNN conserve la structure spatiale grâce à des filtres convolutifs. En pratique, PyTorch implémente une corrélation croisée :

$$S(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n).$$

Le même noyau est appliqué à toutes les positions de l'image. Ce partage des poids permet de détecter le même motif à plusieurs endroits.

II.1.3 Formule dimensionnelle

La taille de sortie d'une convolution ou d'un pooling est calculée par :

$$O = \left\lfloor \frac{I - K + 2P}{S} \right\rfloor + 1,$$

où I est la taille d'entrée, K la taille du noyau, P le padding et S le stride. Cette formule est essentielle pour construire correctement les architectures CNN et éviter les erreurs de dimensions.

II.1.4 Pooling et convolution 1×1

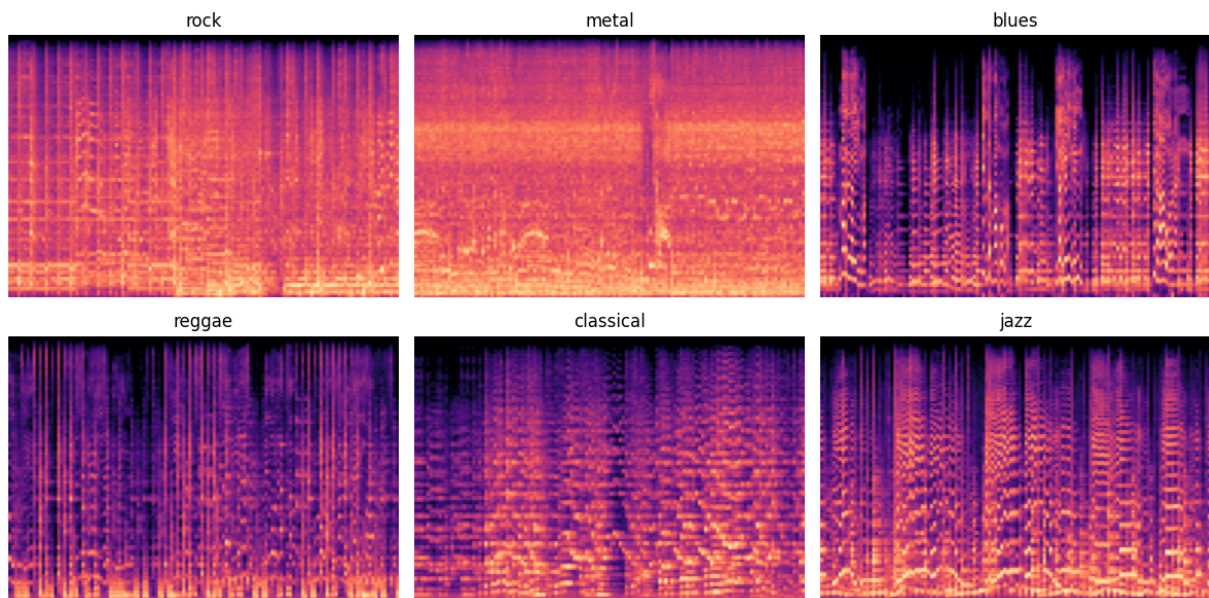
Le max-pooling réduit les dimensions spatiales tout en conservant les activations les plus fortes. La convolution 1×1 agit comme une combinaison linéaire des canaux et permet de modifier le nombre de canaux sans changer la résolution spatiale.

II.2 Cadre Expérimental

La partie image utilise le dataset GTZAN, composé de fichiers audio répartis en dix genres musicaux. Les fichiers audio sont transformés en Mel-spectrogrammes de taille 128×128 , puis utilisés comme images en entrée d'un CNN.

Table 1 – Paramètres expérimentaux du Chapitre II – CNN / GTZAN

Paramètre	Valeur / Description
Dataset	GTZAN Music Genre Classification
Format source	Fichiers audio WAV convertis en Mel-spectrogrammes
Mode d'exécution	FAST_MODE activé
Exemples utilisés	200 spectrogrammes, soit 20 fichiers par genre
Classes	blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, rock
Taille image	128 × 128 pixels, un canal
Split	140 train, 30 validation, 30 test
Batch size	32
Architectures	MLP image aplatie, CNN LeNet baseline et variantes

**Figure 1** – Exemples de Mel-spectrogrammes générés à partir du dataset GTZAN

II.3 Étude des Variantes Architecturales

Plusieurs variantes de CNN ont été testées afin d'observer l'influence du padding, du stride, du pooling, du nombre de filtres et de la convolution 1×1 .

Table 2 – Résultats comparatifs des modèles image sur le set de validation

Modèle	Train loss	Val. loss	Best val. loss	Val. acc.	Epoch	Paramètres
MLP spectrogrammes aplatis	2.2957	2.2147	2.3876	20.00%	2	4 228 746
CNN LeNet baseline	0.9977	1.2918	1.2472	53.33%	14	4 208 970
CNN padding 0	2.1585	2.1014	2.2836	20.00%	2	3 459 402
CNN padding 2	0.4927	1.4028	1.4028	53.33%	15	4 208 970
CNN max pooling	0.9864	1.3039	1.2640	53.33%	14	4 208 970
CNN average pooling	1.3563	1.4264	1.5228	46.67%	13	4 208 970
CNN petit nombre de filtres	2.1687	2.0848	2.2906	30.00%	4	2 101 994
CNN avec convolution 1×1	1.1649	1.2704	1.2917	53.33%	13	4 209 242

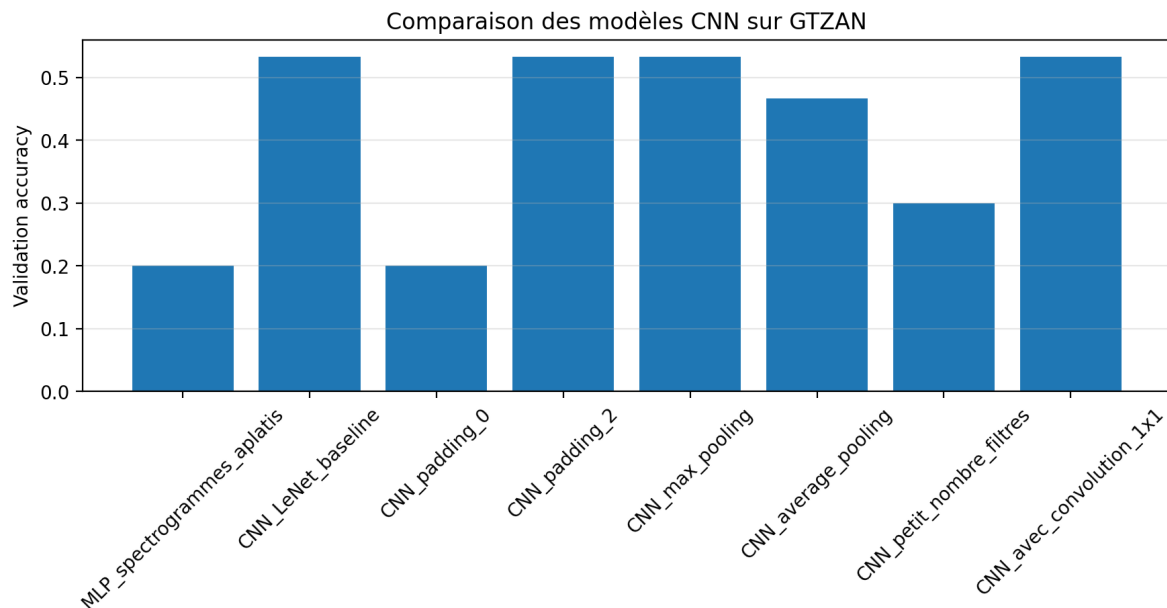


Figure 2 – Comparaison des accuracies de validation des modèles CNN

Analyse et interprétation

Les CNN dépassent nettement le MLP appliqué aux spectrogrammes aplatis. Cela confirme que la structure spatiale d'un spectrogramme contient une information importante : proximité temporelle, bandes fréquentielles et textures sonores. Les variantes avec padding et max-pooling conservent mieux l'information locale. Les résultats restent modestes car l'expérience est réalisée en mode rapide avec seulement 200 exemples.

II.4 Visualisation des Feature Maps

Les cartes de caractéristiques permettent d'observer ce que les premières couches du CNN apprennent. Les couches proches de l'entrée détectent généralement des primitives visuelles : zones d'énergie, transitions fréquentielles et textures. Les couches suivantes combinent ces primitives pour construire des représentations plus abstraites.

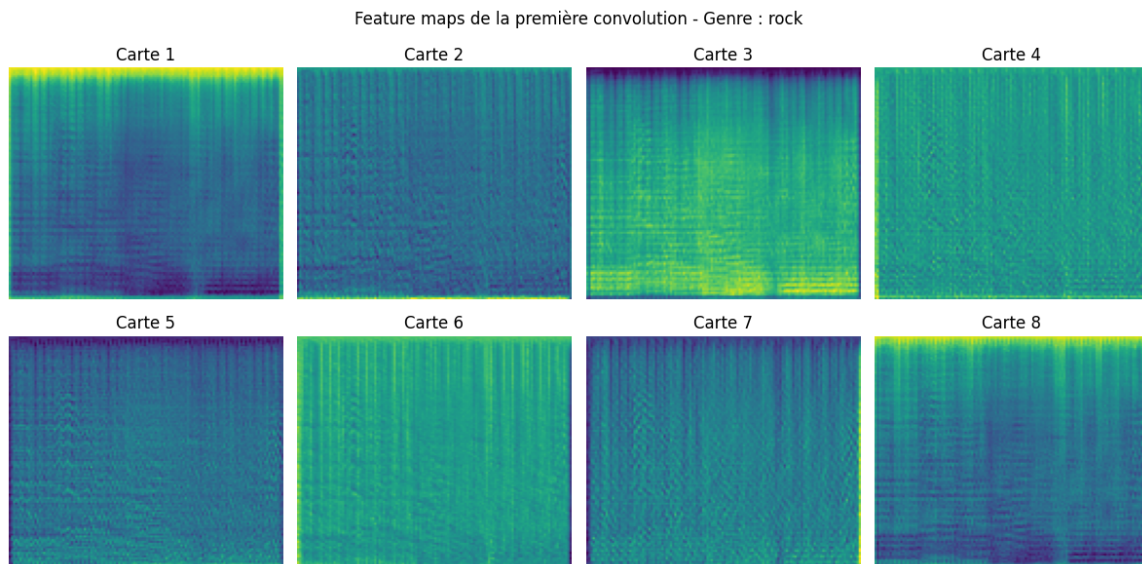


Figure 3 – Visualisation de cartes de caractéristiques apprises par la première convolution du CNN

II.5 Évaluation Finale

Le meilleur modèle CNN atteint une accuracy de test d'environ 50.00%. La matrice de confusion montre que certains genres sont plus facilement distingués, tandis que d'autres restent difficiles à séparer avec un sous-ensemble réduit du dataset.

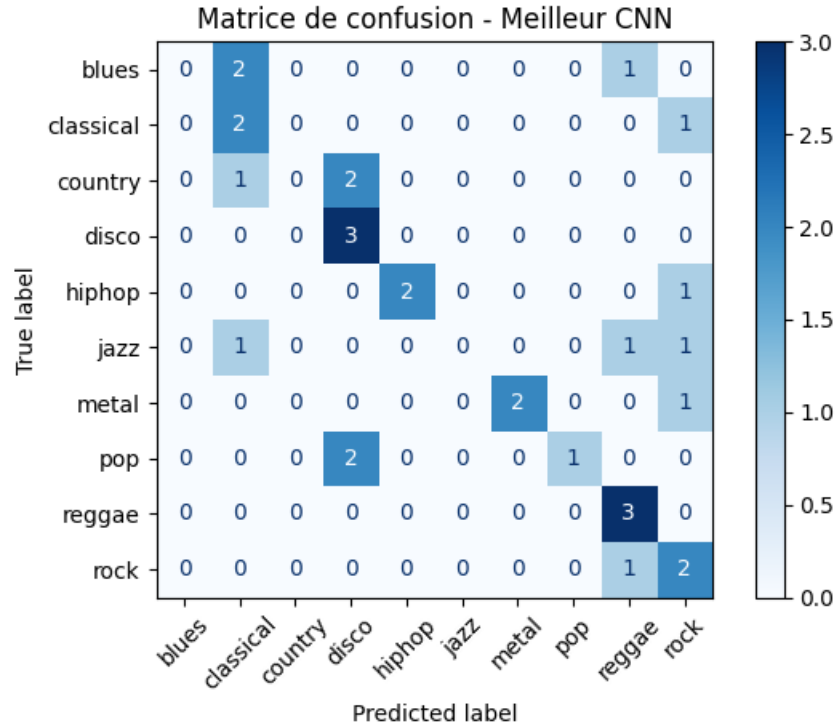


Figure 4 – Matrice de confusion du meilleur modèle CNN sur GTZAN

II.6 Synthèse du Chapitre II

- Les CNN sont plus pertinents que les MLP pour les images car ils conservent la structure spatiale.
- Le padding, le stride et le pooling modifient directement la quantité d'information conservée.
- Augmenter la capacité du CNN peut améliorer les résultats mais augmente le coût de calcul.
- Les feature maps confirment l'apprentissage progressif de représentations visuelles.
- Les performances sont limitées par le mode rapide ; un entraînement sur tout GTZAN serait plus robuste.

Chapitre III

Réseaux Récurrents, LSTM, GRU et Seq2Seq

III.1 Fondements Théoriques et Mathématiques

III.1.1 Limites des architectures statiques face aux séquences

Le MLP et le CNN ne modélisent pas directement la dépendance temporelle entre les éléments d'une séquence. Dans une séquence musicale, l'ordre des notes modifie le motif mélodique et l'interprétation musicale. Il faut donc un modèle capable de conserver une mémoire du passé.

III.1.2 RNN simple

Un RNN introduit un état caché h_t qui dépend de l'entrée courante x_t et de l'état précédent h_{t-1} :

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h).$$

Le même ensemble de poids est partagé à tous les pas de temps, ce qui permet de traiter des séquences de longueur variable.

III.1.3 BPTT et gradient clipping

L'entraînement d'un RNN se fait par rétropropagation à travers le temps. Le gradient traverse plusieurs pas temporels, ce qui peut conduire à des gradients évanescents ou explosifs. Pour limiter l'explosion, on utilise le gradient clipping avec `torch.nn.utils.clip_grad_norm_`.

III.1.4 LSTM et GRU

Le LSTM ajoute une cellule mémoire et des portes d'oubli, d'entrée et de sortie. Le GRU simplifie cette structure avec une porte de mise à jour et une porte de réinitialisation. Dans ce projet, les trois modèles RNN, LSTM et GRU sont comparés sur la prédiction de la prochaine note.

III.1.5 Architecture Seq2Seq

Le modèle Seq2Seq est composé d'un encodeur et d'un décodeur. L'encodeur lit une séquence source et produit un état de contexte. Le décodeur génère une continuation note par note. Pendant l'entraînement, le teacher forcing fournit au décodeur la vraie note précédente. Pendant l'inférence, deux stratégies sont testées : le décodage glouton et le beam search.

III.2 Cadre Expérimental

La partie séquentielle utilise MAESTRO v3.0.0 MIDI-only. Les fichiers MIDI sont parcourus afin d'extraire les hauteurs de notes, triées par temps de début. En mode rapide, 200 fichiers sont utilisés pour construire un corpus de plus d'un million de notes.

Table 1 – Paramètres expérimentaux du Chapitre III – RNN/LSTM/GRU/Seq2Seq

Paramètre	Valeur / Description
Dataset	MAESTRO v3.0.0 MIDI-only
Format	Fichiers MIDI de performances piano
Fichiers trouvés	1 276 fichiers MIDI
Fichiers utilisés	200 fichiers en mode rapide
Notes extraites	1 347 862 notes
Longueur moyenne	6 739.31 notes par séquence
Vocabulaire	91 tokens avec PAD, SOS et EOS
Longueur séquence	50 notes
Split	37 802 train, 8 100 validation, 8 102 test
Modèles	RNN simple, LSTM, GRU et Seq2Seq

III.3 Comparaison des Cellules Récurrentes

Table 2 – Comparaison RNN, LSTM et GRU sur la prédiction de notes

Modèle	Train loss	Val. loss	Perplexité	Best epoch	Paramètres
RNN simple	3.0579	3.0456	21.0234	15	265 435
LSTM	2.6174	2.7110	15.0444	15	956 635
GRU	2.7352	2.7853	16.2046	15	726 235

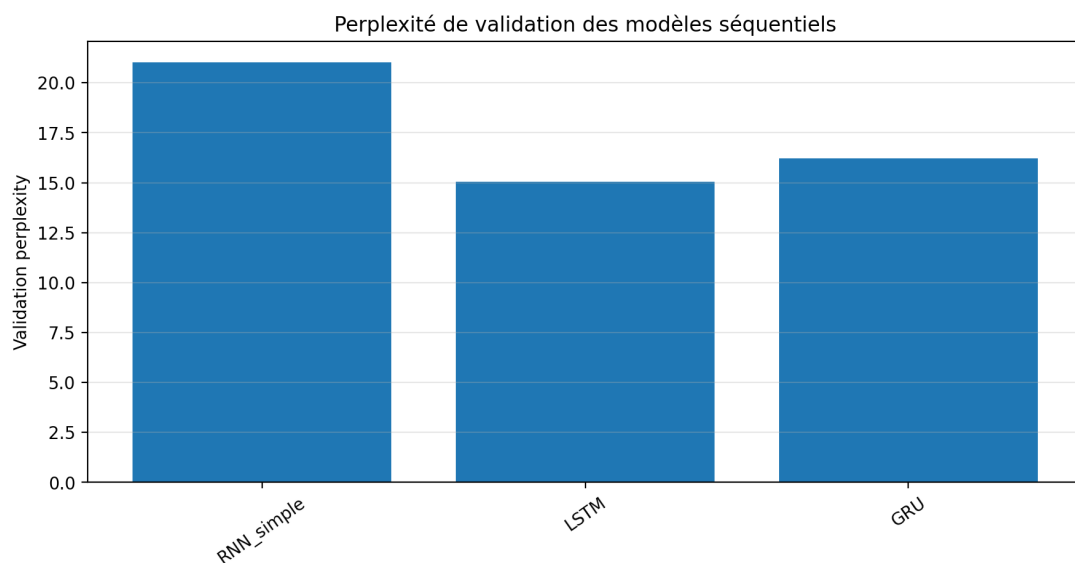


Figure 1 – Comparaison de la perplexité de validation des modèles séquentiels

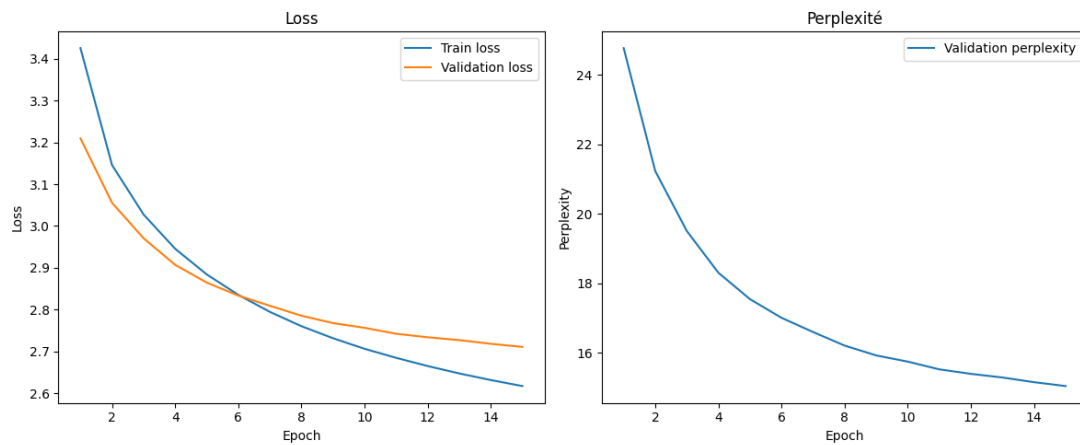


Figure 2 – Courbes d'entraînement du modèle LSTM

Analyse et interprétation

Le LSTM obtient la meilleure perplexité de validation. Ce résultat est cohérent avec son mécanisme de mémoire, qui lui permet de mieux conserver l'information sur plusieurs pas temporels. Le RNN simple est moins performant car il souffre davantage des limites de mémoire et de stabilité du gradient. Le GRU offre un compromis intéressant entre performance et nombre de paramètres.

III.4 Génération Musicale Simple

Le meilleur modèle est le LSTM. Sur le test set, il obtient une loss de 2.7146 et une perplexité de 15.0980. La génération à partir d'une graine de notes montre que le modèle apprend des motifs locaux, mais produit parfois des répétitions.

Exemple de seed test :

[54, 75, 74, 43, 31, 70, 61, 64, 57, 42]

Génération greedy :

[54, 75, 74, 43, 31, 70, 61, 64, 57, 42, 30, 69, 62, 54, 57, 62, 50, 57, 62, 54]

Génération avec beam search :

[54, 75, 74, 43, 31, 70, 61, 64, 57, 42, 30, 69, 62, 42, 30, 69, 62, 57, 30, 42]

III.5 Mini Système Seq2Seq

Le modèle Seq2Seq utilise les 25 premières notes d'une séquence comme entrée et apprend à produire les 25 notes suivantes. La meilleure validation loss obtenue est 3.667. Les sorties générées alternent souvent entre certaines notes fréquentes, ce qui montre que le modèle apprend une continuation probable mais reste limité par une représentation simplifiée des notes.

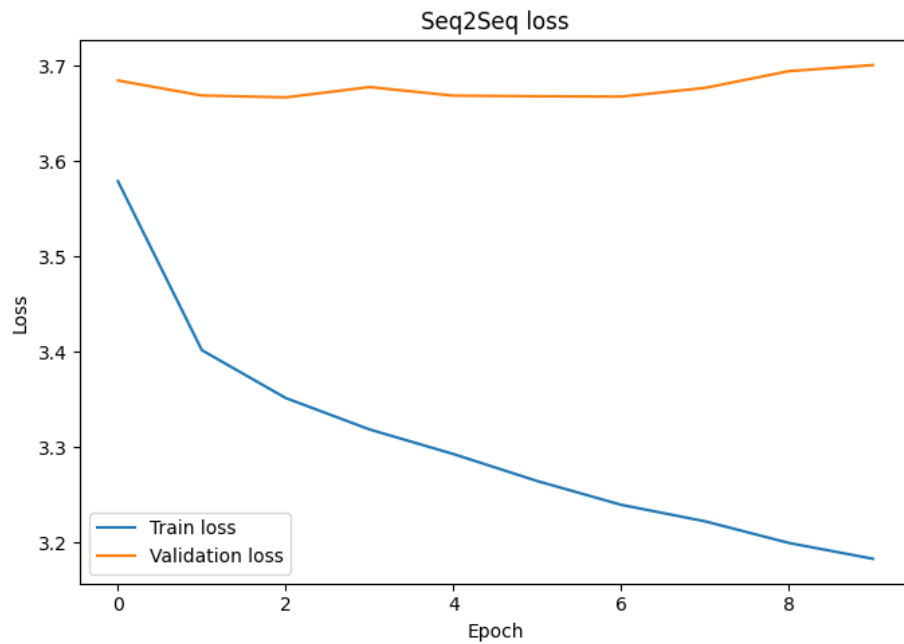


Figure 3 – Courbes d'entraînement du modèle Seq2Seq

III.6 Synthèse du Chapitre III

- Les modèles récurrents exploitent l'ordre temporel des notes grâce à un état caché.
- Le BPTT permet l'entraînement mais peut produire des gradients instables.
- LSTM et GRU améliorent la mémorisation grâce aux portes.
- Le Seq2Seq illustre la génération conditionnelle de continuations musicales.
- Les résultats sont encourageants mais limités par l'absence de durée, de vélocité et d'information harmonique explicite.

Chapitre IV

Comparaison Croisée et Conclusion Générale

IV.1 Tableau Comparatif Global des Trois Architectures

Table 1 – Comparaison globale des architectures étudiées dans MusicAI

Critère	MLP	CNN	RNN / LSTM / GRU / Seq2Seq
Type de données	Tabulaires	Images 2D	Séquences
Dataset utilisé	Spotify Tracks	GTZAN spectrogrammes	MAESTRO MIDI
Biais inductif	Aucun biais spatial ou temporel explicite	Localité, partage des poids, hiérarchie visuelle	Mémoire temporelle, ordre des notes
Meilleure performance	Accuracy test : 61.60%	Accuracy test : 50.00% en mode rapide	Perplexité test LSTM : 15.10
Limite principale	Genres proches difficiles à séparer	Sous-ensemble réduit, test set petit	Génération répétitive sans durée/vélocité
Évolution naturelle	Feature engineering, modèles tabulaires avancés	Data augmentation, ResNet, Vision Transformer	Attention, Transformer, représentation note+durée

IV.2 Le Biais Inductif : Clé de la Réussite en Deep Learning

La comparaison montre que la performance d'un modèle dépend de son adéquation avec la structure des données. Le MLP est raisonnable pour les tableaux mais ignore les relations spatiales. Le CNN exploite la géométrie des spectrogrammes et améliore les résultats par rapport à un MLP appliqué à des pixels aplatis. Le RNN, le LSTM et le GRU introduisent une mémoire séquentielle indispensable pour les suites de notes.

Cette idée rejoint le principe selon lequel aucun algorithme n'est optimal pour tous les problèmes. Le bon modèle est celui dont les hypothèses correspondent le mieux aux régularités de la donnée. Dans MusicAI, les expériences illustrent cette idée de manière concrète : le CNN surpasse le MLP sur images, tandis que les modèles récurrents conviennent mieux aux séquences.

IV.3 Perspectives d'Amélioration

- entraîner la partie CNN sur tout le dataset GTZAN au lieu du mode rapide ;
- appliquer de la data augmentation sur les spectrogrammes ;
- intégrer la durée, la vélocité et les accords dans les tokens MIDI ;
- ajouter un mécanisme d'attention dans le modèle Seq2Seq ;
- comparer les modèles récurrents avec un Transformer musical simple.

IV.4 Conclusion Générale

Ce projet a permis de mettre en pratique les principales familles d'architectures de Deep Learning étudiées dans le module. La partie MLP a montré l'importance de la préparation des données, de l'initialisation, de la gestion des paramètres et de la sauvegarde avec `state_dict`. La partie CNN a confirmé la pertinence des convolutions pour traiter des spectrogrammes musicaux, avec une amélioration par rapport au MLP image aplatie. La partie RNN/Seq2Seq a illustré la modélisation de séquences, l'intérêt des cellules récurrentes et les stratégies de décodage.

La conclusion principale est que le Deep Learning doit adapter son architecture à la nature de la donnée. Les données tabulaires, les images et les séquences ne possèdent pas la même géométrie ni les mêmes dépendances. Utiliser le même paradigme supervisé ne signifie donc pas utiliser le même réseau. Le choix du bon biais inductif reste un facteur central de réussite.

Références Bibliographiques

1. Rumelhart, D. E., Hinton, G. E., Williams, R. J. (1986). *Learning representations by back-propagating errors*. Nature.
2. Cybenko, G. (1989). *Approximation by superpositions of a sigmoidal function*. Mathematics of Control, Signals and Systems.
3. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P. (1998). *Gradient-based learning applied to document recognition*. Proceedings of the IEEE.
4. Hochreiter, S., Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation.
5. Cho, K. et al. (2014). *Learning phrase representations using RNN encoder-decoder*. EMNLP.
6. Glorot, X., Bengio, Y. (2010). *Understanding the difficulty of training deep feedforward neural networks*. AISTATS.
7. Kingma, D. P., Ba, J. (2015). *Adam : A method for stochastic optimization*. ICLR.
8. Paszke, A. et al. (2019). *PyTorch : An imperative style, high-performance deep learning library*. NeurIPS.
9. Spotify Tracks Dataset, Kaggle.
10. GTZAN Music Genre Classification Dataset.
11. MAESTRO Dataset v3.0.0, Magenta / Google.